

Decoder-Focused Encoder-Decoder + Attention

A consolidated, decoder-first set of notes combining the previous discussions: encoder/decoder states, teacher forcing, inference, Bahdanau attention, dimensions, Softmax, context vectors, the second-word prediction, and the crucial distinction between one LSTM layer and its repeated timesteps.

0. The one picture to remember

Encoder creates the information bank H. Decoder creates one target word at a time. At every decoder timestep, the same LSTM layer is reused; its hidden state S and cell state C carry decoder history, while attention creates a fresh Context vector from the encoder outputs.

Encoder		Decoder — repeated across time
Source tokens I → love → you	→	Encoder outputs $H_1, H_2, H_3 \dots$
Final encoder states h_{enc}, c_{enc}	→	Projection $h_{enc} \rightarrow S_0$ $c_{enc} \rightarrow C_0$
	→	Time t $X_t + S_{t-1} + C_{t-1} \rightarrow \text{LSTM} \rightarrow S_t, C_t$
	→	$S_t \rightarrow \text{Attention}(H_1 \dots H_n) \rightarrow \text{Context}_t$
	→	$[S_t ; \text{Context}_t] \rightarrow \text{Dense} \rightarrow \text{Softmax} \rightarrow \hat{y}_t$
	→	Training: correct token → Embedding → X_{t+1} Inference: predicted token → Embedding → X_{t+1}

1. Fixed architecture and dimensions

To avoid the dimension confusion we discussed, keep one architecture fixed throughout this example.

Component	Configuration	Dimension
Encoder	Bidirectional LSTM: 256 forward + 256 backward	$H_i = 512$
Encoder final hidden	forward + backward concatenated	512
Encoder final cell	forward + backward concatenated	512
Decoder	Normal LSTM	256 units
Decoder initial hidden S_0	$512 \rightarrow \text{Dense/Linear projection}$	256
Decoder initial cell C_0	$512 \rightarrow \text{Dense/Linear projection}$	256
Token embedding X_t	Example embedding layer	128
Attention space	Learned projections	128
Context_t	Weighted sum of encoder H vectors	512

2. Encoder output vs decoder initial state

With a 256-unit BiLSTM, each encoder output is 256 forward + 256 backward = 512-D. The final encoder hidden and cell states are also 512-D after combining the two directions.

But the decoder has only 256 units. Therefore the encoder states are projected separately:

encoder final hidden [512] → Dense → S_0 [256]

encoder final cell [512] → Dense → C_0 [256]

Do not concatenate hidden and cell into 1024 and call that one decoder state. H and C are different LSTM states.

3. What exactly enters one decoder LSTM step?

At decoder timestep t , there are three inputs to the LSTM operation:

1. X_t — embedding of the current decoder input token.
2. S_{t-1} — previous decoder hidden state.
3. C_{t-1} — previous decoder cell state.

The LSTM produces S_t and C_t together. S_t does not first get generated and then somehow generate C_t . Both come from the same LSTM cell computation.

$$(X_t, S_{t-1}, C_{t-1}) \rightarrow \text{Decoder LSTM} \rightarrow (S_t, C_t)$$

4. The first decoder timestep

At inference, the decoder begins with SOS. At training, the first decoder input is also normally SOS.

$$\text{SOS} \rightarrow \text{Embedding} \rightarrow X_1$$

$$(X_1, S_0, C_0) \rightarrow \text{LSTM} \rightarrow S_1, C_1$$

Now attention uses the decoder state to inspect all encoder outputs and create Context_1 .

$$S_1 + H_1, H_2, H_3, \dots \rightarrow \text{Attention} \rightarrow \text{Context}_1$$

Then the output layer uses S_1 and Context_1 to predict the first target token:

$$[S_1 ; \text{Context}_1] \rightarrow \text{Dense} \rightarrow \text{vocabulary logits} \rightarrow \text{Softmax} \rightarrow \hat{y}_1$$

5. The most important distinction: S_1 , C_1 and Context_1

Object	What it means	Where it goes next
S_1	Decoder hidden state; current decoder representation	Used for attention; also combined with Context_1 for output; then enters the next LSTM step
C_1	Decoder cell state / internal memory	Passed directly to the next LSTM step as previous cell state
Context_1	Fresh encoder information selected by attention for this timestep	Used in the current output prediction; next timestep creates Context_2
X_2	Embedding of the next decoder input token	Enters the next LSTM step

Mental model: C travels through the LSTM. S talks to attention. Context brings encoder information to the output layer. X is the current token embedding.

6. One word prediction does NOT mean two LSTM nodes

This was one of the main doubts in our discussion. There is one decoder LSTM layer. The layer is executed repeatedly at different timesteps. The weights are shared.

Time 1	Time 2	Time 3
X_1, S_0, C_0	X_2, S_1, C_1	X_3, S_2, C_2
↓	↓	↓
SAME LSTM shared weights	SAME LSTM shared weights	SAME LSTM shared weights
↓	↓	↓
S_1, C_1	S_2, C_2	S_3, C_3
↓	↓	↓

Attention → Context ₁ → Word 1	Attention → Context ₂ → Word 2	Attention → Context ₃ → Word 3
--	--	--

Therefore, one predicted word corresponds to one time-step execution of the same decoder LSTM, followed by attention and the output layer. It does not mean two LSTM layers.

7. Bahdanau attention — what happens after S_1 ?

For the decoder's current state S_1 , compare it with every encoder output H_i . Because S_1 is 256-D and H_i is 512-D, project both into a common 128-D attention space.

$$W_s S_1 \rightarrow 128\text{-D}$$

$$W_h H_i \rightarrow 128\text{-D}$$

$$E_{1,i} = \tanh(W_s S_1 + W_h H_i + b) \rightarrow 128\text{-D vector}$$

Then a learned scoring vector v_a converts each E vector into one scalar alignment score:

$$e_{1,i} = v_a^T E_{1,i} \rightarrow \text{scalar}$$

Important: v_a is a learned parameter vector, not three independent neural networks. The same attention-scoring parameters are shared across all encoder positions.

8. E vectors, alignment scores and attention weights are different things

Symbol	Shape	Meaning
$E_{1,1} / E_{1,2} / E_{1,3}$	128-D vector	Intermediate attention representation
$e_{1,1} / e_{1,2} / e_{1,3}$	scalar	Raw alignment score
$A_{1,1} / A_{1,2} / A_{1,3}$	scalar	Softmax attention weight
$H_1 / H_2 / H_3$	512-D vector	Encoder output at each source position
Context ₁	512-D vector	Weighted sum of all encoder outputs

9. Softmax: do we take only the maximum attention?

No. Softmax is applied across all alignment scores for the current decoder timestep:

$$A_1 = \text{Softmax}([e_{1,1}, e_{1,2}, e_{1,3}])$$

Example: if the scores produce $A_1 = [0.341, 0.335, 0.324]$, we use all three weights. We do not take only 0.341.

10. Weighted sum: why a scalar attention weight multiplies a vector

Each $A_{1,i}$ is a scalar and each H_i is a vector. Therefore scalar $\times$ vector gives another vector of the same dimension:

$$\text{Context}_1 = A_{1,1}H_1 + A_{1,2}H_2 + A_{1,3}H_3$$

If every H_i is 512-D, Context₁ is also 512-D. The attention weights decide how much each encoder vector contributes.

11. Numerical toy example

For arithmetic, use 3-D vectors instead of the real 256/512/128 dimensions. The operations are identical.

$$S_1 = [0.5, 0.2, 0.1]$$

$$H_1 = [0.8, 0.1, 0.4]$$

$$H_2 = [0.2, 0.7, 0.3]$$

$$H_3 = [0.1, 0.3, 0.9]$$

Toy assumption: projected dimensions already match and $E_{1,i} = \tanh(S_1 + H_i)$.

$$E_{1,1} = \tanh([1.3, 0.3, 0.5]) \approx [0.862, 0.291, 0.462]$$

$$E_{1,2} = \tanh([0.7, 0.9, 0.4]) \approx [0.604, 0.716, 0.380]$$

$$E_{1,3} = \tanh([0.6, 0.5, 1.0]) \approx [0.537, 0.462, 0.762]$$

Let $v_a = [0.5, 0.3, 0.2]$. Then:

$$e_{1,1} \approx 0.6107; e_{1,2} \approx 0.5928; e_{1,3} \approx 0.5595$$

$$\text{Softmax}([0.6107, 0.5928, 0.5595]) \approx [0.341, 0.335, 0.324]$$

$$\text{Context}_1 = 0.341H_1 + 0.335H_2 + 0.324H_3 \approx [0.3722, 0.3658, 0.5285].$$

12. Now the crucial part: how is the FIRST word predicted?

After the LSTM and attention, we have S_1 and Context_1 :

$$S_1 = 256\text{-D}$$

$$\text{Context}_1 = 512\text{-D}$$

Concatenate them: $[S_1 ; \text{Context}_1] = 768\text{-D}$. Then the output projection produces one logit for every vocabulary token.

$$[S_1 ; \text{Context}_1] \rightarrow \text{Dense}(V) \rightarrow V \text{ logits} \rightarrow \text{Softmax} \rightarrow V \text{ probabilities}$$

If vocabulary size $V = 10,000$, the final Softmax has 10,000 probabilities. This is large, but it is standard in a normal sequence-to-sequence model.

13. How is the SECOND word predicted?

Suppose the first predicted word is main. During inference, convert that predicted token to its embedding:

$$\text{main} \rightarrow \text{Embedding} \rightarrow X_2$$

Now the previous decoder states are still available as S_1 and C_1 . Therefore:

$$(X_2, S_1, C_1) \rightarrow \text{SAME Decoder LSTM} \rightarrow S_2, C_2$$

Use S_2 to calculate a completely new attention distribution:

$$S_2 + H_1, H_2, H_3 \rightarrow \text{Attention} \rightarrow \text{Context}_2$$

Then predict the second word:

$$[S_2 ; \text{Context}_2] \rightarrow \text{Dense} \rightarrow \text{Softmax} \rightarrow \text{word 2}$$

For example: main $\rightarrow$ tumse.

14. What happened to Context_1 ?

Context_1 is not passed as the next LSTM state. It was the encoder summary specifically retrieved for timestep 1. At timestep 2, attention is recalculated and creates Context_2 .

So the carried information is:

$$S_1 + C_1 \rightarrow \text{next LSTM}$$

while the newly generated information is:

$$S_2 \rightarrow \text{Attention} \rightarrow \text{Context}_2$$

15. Full inference example

Source: I love you

Target example: main tumse pyaar karta hoon

Encoder: I $\rightarrow$ love $\rightarrow$ you $\rightarrow H_1, H_2, H_3$ + final encoder states $\rightarrow S_0, C_0$

Step 1: SOS $\rightarrow X_1 \rightarrow \text{LSTM}(S_0, C_0) \rightarrow S_1, C_1 \rightarrow \text{Attention} \rightarrow \text{Context}_1 \rightarrow \text{Softmax} \rightarrow \text{main}$

Step 2: main $\rightarrow X_2 \rightarrow \text{SAME LSTM}(S_1, C_1) \rightarrow S_2, C_2 \rightarrow \text{Attention} \rightarrow \text{Context}_2 \rightarrow \text{Softmax} \rightarrow \text{tumse}$

Step 3: tumse $\rightarrow X_3 \rightarrow \text{SAME LSTM}(S_2, C_2) \rightarrow S_3, C_3 \rightarrow \text{Attention} \rightarrow \text{Context}_3 \rightarrow \text{Softmax} \rightarrow \text{pyaar}$

Step 4: pyaar $\rightarrow X_4 \rightarrow \dots \rightarrow \text{karta}$

Step 5: karta $\rightarrow X_5 \rightarrow \dots \rightarrow \text{hoon}$

Step 6: hoon $\rightarrow X_6 \rightarrow \dots \rightarrow \text{EOS} \rightarrow \text{stop}$

16. Training vs inference — the exact distinction

Training	Inference
Target sequence is known.	Target sequence is unknown.
Teacher forcing is commonly used.	No teacher forcing from ground truth.
At step t , use the correct previous target token to make X_t .	At step t , use the model's previous predicted token to make X_t .
$\hat{y}_t$ is compared with Y_t using cross-entropy.	There is no Y_t available for the new example.
All target positions can often be processed efficiently in parallel.	Generation is autoregressive: token by token.

17. Teacher forcing with the example sentence

Target: <START> main tumse pyaar karta hoon <END>

Decoder input $\rightarrow$ Expected output

<START> $\rightarrow$ main

main $\rightarrow$ tumse

tumse $\rightarrow$ pyaar

pyaar $\rightarrow$ karta

karta $\rightarrow$ hoon

hoon $\rightarrow$ <END>

During training, if the model predicted the wrong first word, the next training input can still be the correct ground-truth token 'main'. That is the essence of teacher forcing.

18. What exactly is passed to the next step?

Do not say: ' H_t is passed to the next LSTM.' That is incorrect for the decoder.

Say instead:

The previous decoder hidden and cell states are carried forward, while the previous token prediction is embedded and supplied as the next decoder input.

So the next decoder step receives exactly:

1. Previous decoder state: S_{t-1}, C_{t-1}

2. Current token embedding: X_t

19. Final master formula

(X_t, S_{t-1}, C_{t-1})

$\downarrow$

Decoder LSTM

$\downarrow$

S_t, C_t

$\downarrow$

$S_t + \text{Encoder } H \rightarrow \text{Bahdanau Attention} \rightarrow \text{Context}_t$

$\downarrow$

$[S_t; \text{Context}_t] \rightarrow \text{Dense}(V) \rightarrow \text{Softmax} \rightarrow \hat{y}_t$

$\downarrow$

Choose token $\rightarrow$ Embedding $\rightarrow X_{t+1}$

20. The five distinctions you should be able to explain without notes

1. One decoder LSTM layer $\neq$ one LSTM node per word. The same layer is reused across timesteps.
2. S_t and C_t are produced together by the LSTM; S_t is not used to create C_t afterward.
3. C_t is carried to the next LSTM step; Context_t is recalculated by attention at every timestep.
4. Attention weights are scalars and encoder outputs are vectors; their weighted sum remains a vector.
5. During inference, the predicted token is embedded and fed back; S and C are also carried forward.

21. One-line memory aid

Token comes in $\rightarrow$ LSTM updates $S/C \rightarrow S$ asks attention where to look $\rightarrow$ Context brings encoder information $\rightarrow S + \text{Context}$ predict a word $\rightarrow$ predicted word becomes the next token.